

SubTrack Project Master Context Document

Version: September 2026

Author: Harsh Chauhan

Purpose: Project Continuity Document

This document contains the complete context of the SubTrack project so that any AI assistant (ChatGPT, Claude, Copilot, Gemini, etc.) can immediately understand the architecture, features implemented, decisions made, current status, and future roadmap.

1. Project Overview

Project Name

SubTrack

Purpose

SubTrack is a subscription management platform that allows users to:

- Track subscriptions
 - View upcoming renewals
 - Manage subscription statuses
 - Process payments
 - Receive notifications
 - Automatically renew subscriptions using a background worker
-

2. Architecture

The project follows a **Microservice Architecture**.

Services

Authentication Service

Responsible for:

- Registration
- Login
- JWT Generation
- User Management
- Roles

Database:

Plain Text

1

AuthenticationDb

Show more lines

Subscription Service

Responsible for:

- Create Subscription
- Update Subscription
- Delete Subscription
- Get Subscriptions
- Manage Status
- Due Subscription Retrieval
- Subscription Renewal

Database:

Plain Text

1

SubscriptionDb

Show more lines

Payment Service

Responsible for:

- Payment Processing

- Payment History
- Simulated Payment Status Generation

Database:

Plain Text

1

PaymentDb

Show more lines

Notification Service

Responsible for:

- Notification Creation
- Notification Retrieval
- Mark Read
- Mark All Read
- Unread Count

Database:

Plain Text

1

NotificationDb

Show more lines

Renewal Worker

Background service which:

Plain Text

1

Gets Due Subscriptions

2

↓

3

Processes Payment

4

↓

5

Creates Notifications

6

↓

7

Renews Subscription

Show more lines

3. Authentication

JWT Authentication

Implemented using:

C#

1

ClaimTypes.Nameldentifier

2

ClaimTypes.Role

Show more lines

Claims included:

C#

1

new Claim(

2

ClaimTypes.Nameldentifier,

3

user.Id.ToString()

4

)

5

6

new Claim(

7

ClaimTypes.Role,

8

user.Role.ToString()

9

)

Show more lines

Roles

Plain Text

1

User

2

Worker

3

Admin

Show more lines

JWT Helper

Created helper:

C#

1

private Guid GetCurrentUserId()

Show more lines

Purpose:

Plain Text

1

Avoid repeating JWT parsing logic

Show more lines

Usage:

C#

1

var userId = GetCurrentUserId();

Show more lines

4. Subscription Module

Features

Create Subscription

Validates:

Plain Text

1

No duplicate:

2

- Name

3

- Category

4

- UserId

Show more lines

Returns:

C#

1

SubscriptionResponseDto

Show more lines

Update Subscription

Supports partial update.

Fields:

Plain Text

1

Name

2

Amount

3

Category

4

BillingCycle

5

NextBillingDate

Show more lines

Update Status

Statuses:

C#

1

Active

2

Paused

3

Cancelled

Show more lines

Endpoint:

Plain Text

1

PATCH

Show more lines

Reason:

Plain Text

1

Partial Update

Show more lines

Get User Subscriptions

Returns:

Plain Text

1

Active

2

Paused

Show more lines

Excludes:

Plain Text
1
Cancelled
Show more lines

Get Categories

Distinct categories.

Delete Subscription

Deletes subscription by ID.

5. BillingCycle

C#
1
Monthly
2
Yearly
Show more lines

Used during renewal.

6. Renewal Logic

Implemented in:

C#
1
RenewSubscription()
Show more lines

Logic:

Monthly

C#

1

AddMonths(1)

Show more lines

Yearly

C#

1

AddYears(1)

Show more lines

Validation:

Plain Text

1

Subscription Exists

2

Must Be Active

3

Cannot Be Cancelled

Show more lines

7. Payment Module

Payment Entity

Contains:

Plain Text

1

Id

2

UserId

3

SubscriptionId

4

Amount

5
Status
6
TransactionReference
7
PaymentDate
Show more lines

Payment Status

Implemented Enum:

C#
1
Pending
2
Completed
3
Failed
4
Cancelled
Show more lines

Payment status is currently simulated:

C#
1
Status =
2
(PaymentStatus)Random.Shared.Next(0,4);
Show more lines

Transaction Reference

Generated:

C#
1
TXN-{{Guid}}
Show more lines

Example:

Plain Text

1

TXN-8f39d8a....

Show more lines

8. Important Payment Design Change

Initially:

Plain Text

1

Worker considered:

2

3

HTTP 200

4

=

5

Payment Success

Show more lines

This was wrong.

Example:

HTTP

1

200 OK

Show more lines

could still contain:

JSON

1

{

2

"status": "Failed"

3

```
}
```

Show more lines

Final Fix Implemented

Worker now reads:

C#

1

PaymentResponseDto

Show more lines

and checks:

C#

1

paymentResponse.Status ==

2

PaymentStatus.Completed

Show more lines

Only then:

Plain Text

1

Renew Subscription

Show more lines

This is an important architectural improvement.

9. Strategy Pattern Refactor

Problem:

Normal users and Worker process payments differently.

User:

Plain Text

1
UserId from JWT
Show more lines

Worker:

Plain Text
1
UserId from DTO
Show more lines

Solution

Created:

C#
1
IProcessPaymentDto
Show more lines

Implemented by:

C#
1
ProcessPaymentDto
Show more lines

and

C#
1
WorkerProcessPaymentDto
Show more lines

Shared internal method:

C#
1
CreatePayment(
2
IProcessPaymentDto dto,
3

Guid userId)
Show more lines

Benefits:

Plain Text
1
DRY
2
Safer
3
Cleaner design
Show more lines

10. Notification Module

Features

Get My Notifications

HTTP
1
GET
2
/notification/my
Show more lines

Mark Read

HTTP
1
PATCH
2
/notification/read/{id}
Show more lines

Mark All Read

HTTP

1

PATCH

2

/notification/readall

Show more lines

Unread Count

HTTP

1

GET

2

/notification/unreadcount

Show more lines

11. Renewal Worker Flow

Current Worker:

Plain Text

1

Get Due Subscriptions

2

↓

3

Process Payment

4

↓

5

If Completed

6

↓

7

Renew Subscription

8

↓

9

Create Success Notification

Show more lines

Else:

Plain Text

1

Create Failure Notification

Show more lines

12. Frontend

Built using:

Plain Text

1

React

2

TypeScript

3

Tailwind CSS

4

Axios

Show more lines

13. Frontend Pages

Implemented

Dashboard

Shows:

Plain Text

1

Summary

2

Statistics

3

Subscription Data

Show more lines

Subscriptions

Plain Text

1

Create

2

Update

3

Delete

4

View

Show more lines

Payments

Plain Text

1

Payment History

Show more lines

Notifications

Implemented:

Plain Text

1

List Notifications

2

Mark Read

3

Mark All Read

4

Auto Refresh

5

Unread Highlight

Show more lines

14. Sidebar Notification Badge

Implemented.

Example:

Plain Text

1

Notifications (5)

Show more lines

Uses:

HTTP

1

GET

2

/notification/unreadcount

Show more lines

Refreshes every:

Plain Text

1

15 seconds

Show more lines

using:

C#

1

setInterval()

Show more lines

15. Notifications Page Auto Refresh

Implemented polling:

TypeScript

1

```
setInterval(  
  2  
  loadNotifications,  
  3  
  15000  
  4  
);  
Show more lines
```

Purpose:

Plain Text
1
Worker creates notification
2
Page updates automatically
Show more lines

without refresh.

16. Service Layer Refactor

Originally:

Plain Text
1
Controller
2
↓
3
Business Logic
4
↓
5
DbContext
Show more lines

Refactored to:

Plain Text

1
Controller
2
↓
3
Service
4
↓
5
DbContext
Show more lines

Example

Created:

C#
1
IPaymentService
Show more lines
C#
1
PaymentService
Show more lines

and similar services.

17. Repository Pattern Decision

Decision made:

Plain Text
1
NO Repository Layer
Show more lines

Current architecture:

Plain Text

1
Controller
2
↓
3
Service
4
↓
5
EF DbContext
Show more lines

Reason:

Plain Text
1
EF Core already acts as
2
Repository + Unit Of Work
Show more lines

Avoids unnecessary complexity.

18. Global Exception Handling

Recently implemented.

Created:

C#
1
GlobalExceptionHandler
Show more lines

Custom Exceptions

Examples:

C#

1
BadRequestException
2
NotFoundException
3
UnauthorizedException
Show more lines

Authentication:

C#
1
UserEmailAlreadyExistsException
Show more lines

Returns:

HTTP
1
409 Conflict
Show more lines

Middleware Responses

Not Found

JSON
1
{
2
"message": "Subscription not found",
3
"statusCode": 404
4
}
Show more lines

Bad Request

JSON

```
1
{
2
"message":"Invalid request",
3
"statusCode":400
4
}
Show more lines
```

Conflict

```
JSON
1
{
2
"message":"Email already exists",
3
"statusCode":409
4
}
Show more lines
```

Internal Server Error

```
JSON
1
{
2
"message":"An unexpected error occurred",
3
"statusCode":500
4
}
Show more lines
```

19. Important Lessons Learned During Development

JWT Is Stateless

Multiple logins:

Plain Text

1

React

2

Postman

3

Worker

Show more lines

all produce valid tokens.

Old tokens remain valid until expiry.

ToListAsync()

Returns:

C#

1

[]

Show more lines

Never:

C#

1

null

Show more lines

FirstOrDefaultAsync()

Returns:

C#

1

null

Show more lines

when no record found.

PATCH vs PUT

Decision:

Plain Text

1

Role Change

2

Subscription Status Update

Show more lines

Should use:

HTTP

1

PATCH

Show more lines

because they are partial updates.

20. Current Project Status

Backend

Completed

- ✓ Authentication
 - ✓ Authorization
 - ✓ JWT
 - ✓ Subscription Module
 - ✓ Payment Module
 - ✓ Notification Module
 - ✓ Renewal Worker
 - ✓ Service Layer
 - ✓ Exception Middleware
-

Frontend

Completed

- ✓ Dashboard
 - ✓ Subscription Pages
 - ✓ Payment Pages
 - ✓ Notification Center
 - ✓ Notification Counter
 - ✓ Auto Refresh
-

21. Remaining Work

High Priority

Finish Service Refactoring

Create services for:

Plain Text

1

Auth

2

Payments

3

Notifications

4

Subscriptions

Show more lines

if anything remains.

Replace Entity Returns

Current issue:

Some APIs still return:

Plain Text

1

Entities

Show more lines

Need:

Plain Text

1

DTOs only

Show more lines

Complete Global Exception Adoption

Replace:

C#

1
ArgumentException
Show more lines

with proper custom exceptions.

Configure Decimal Precision

Current EF warning:

C#
1
Balance
Show more lines

Needs:

C#
1
.HasPrecision(18,2)
Show more lines

22. Future Enhancements

Possible next features:

Plain Text
1
Notification Icons
2
Unread Filter
3
Dashboard Charts
4
Budget Management
5
Refresh Tokens
6
Email Notifications

7

SignalR Real-Time Notifications

8

Admin Portal

Show more lines

23. Important Reminder

A previously noted improvement was completed:

Originally:

Plain Text

1

Worker renewed subscriptions

2

based only on HTTP success.

Show more lines

Now:

Plain Text

1

Worker uses

2

PaymentResponseDto.Status

Show more lines

and renews only when:

C#

1

PaymentStatus.Completed

Show more lines

This is a key architectural decision and should not be reverted.

Final Project State

SubTrack is currently a nearly complete microservice-based subscription management platform featuring:

- JWT Authentication
- Role-Based Authorization
- Subscription Management
- Payment Processing
- Notification System
- Renewal Worker
- React Frontend
- Service Layer Architecture
- Global Exception Handling
- Separate Database per Service

The project is currently in the **polish, cleanup, and production-readiness phase**, not the feature-building phase. The next focus should be architecture cleanup, DTO consistency, exception standardization, and final UX improvements.